

DIO · FORMAÇÃO CHATGPT FOR DEVS

O DEV AUMENTADO

Claude Code na prática: da ideia ao commit

Como a IA agêntica encurta o caminho entre a ideia e o código funcional



claude-code

```
$ claude  
> crie o endpoint /perfil com testes
```

```
● Read   src/routes/routes.js  
● Edit   src/controllers/perfil.js  
● Write  tests/perfil.test.js  
● Bash   npm test    12 passed
```

Endpoint criado e testes passando.

Filipe Zanin

Desafio de projeto · DIO

SUMÁRIO

01	Da sugestão à ação	4
	O que muda quando a IA tem mãos	
02	Onboarding	6
	Entendendo uma base que você nunca viu	
03	Ponta a ponta	9
	Funcionalidade completa, com testes	
04	Debugging	12
	Da mensagem de erro à causa raiz	
05	Refatoração	15
	Mudanças mecânicas, em escala	
06	Memória de projeto	17
	Ensinando as regras do seu time	
07	Acelerador	20
	Não substituto	

Este e-book foi produzido como desafio de projeto da DIO — Formação ChatGPT for Devs.

O texto foi acelerado por IA e revisado, editado e diagramado por um humano. A diagramação é gerada por código: todas as páginas que você está lendo saem de um script Python versionado no repositório.

ANTES DE COMEÇAR

Existe uma diferença grande entre uma IA que sugere e uma IA que executa. A primeira te devolve um trecho de código para você copiar. A segunda abre o seu repositório, lê os arquivos, entende as convenções do projeto, edita o que precisa ser editado e roda os testes para conferir se não quebrou nada.

Esse segundo grupo tem nome: ferramentas agênticas. E o Claude Code, da Anthropic, é uma delas. Este e-book é sobre o que muda no dia a dia de quem programa quando a IA sai da aba do navegador e entra no terminal, no editor e no fluxo de trabalho.

Não é um manual de referência — a documentação oficial faz isso melhor. É um guia de postura: onde essas ferramentas ganham tempo de verdade, onde elas atrapalham, e o que continua sendo, inegociavelmente, trabalho seu.

O ganho não está em terceirizar o pensamento. Está em encurtar a distância entre a ideia e o código funcional.

Cada capítulo traz uma situação concreta, o que você pediria e o que acontece na prática. Leia na ordem ou pule direto para o problema que você tem hoje.

CAPÍTULO

01

DA SUGESTÃO À AÇÃO

O que muda quando a IA tem mãos

A IA SAIU DA ABA DO NAVEGADOR

Por anos o fluxo foi sempre o mesmo: descrever o problema num chat, receber um bloco de código, voltar para o editor, colar, adaptar os nomes das variáveis, descobrir que faltava um import. Funcionava, mas o custo de tradução entre a resposta genérica e o seu projeto real era todo seu.

Uma ferramenta agêntica inverte isso. Ela opera dentro do repositório, então não precisa adivinhar como o seu projeto é organizado — ela lê. As três capacidades que fazem a diferença:

- Ler arquivos e navegar pela estrutura do projeto
- Editar arquivos existentes, não só propor trechos
- Executar comandos e reagir ao resultado deles

A terminal window with a dark background and light text. The title bar says 'terminal'. The prompt is '\$ claude'. The user input is '> adicione paginação no endpoint de listagem'. The output shows three actions: 'Read src/routes/produtos.js', 'Edit src/controllers/produtos.js', and 'Bash npm test 18 passed'.

```
terminal

$ claude
> adicione paginação no endpoint de listagem

• Read  src/routes/produtos.js
• Edit  src/controllers/produtos.js
• Bash  npm test 18 passed
```

A diferença não é de qualidade do código gerado. É de onde o trabalho acontece.

CAPÍTULO

02

ONBOARDING

Entendendo uma base que você nunca viu

O CUSTO REAL DE ENTRAR NUM PROJETO

Entrar num repositório desconhecido é, de longe, uma das partes mais lentas do trabalho. Não porque o código seja difícil, mas porque o conhecimento está espalhado: um pouco no README desatualizado, um pouco na cabeça de quem escreveu, um pouco só no histórico do Git.

Aqui a IA agêntica rende muito, porque a tarefa é essencialmente de leitura em volume — exatamente o que uma pessoa faz devagar e uma máquina faz rápido.

Peça o mapa antes de pedir a mudança.

Perguntas que rendem no primeiro dia:

- Qual é o ponto de entrada da aplicação?
- Como a autenticação funciona e onde ela é aplicada?
- Quais são os 5 arquivos que mais mudam neste repo?

DO MAPA AO PRIMEIRO COMMIT

Depois do panorama, o próximo passo é entender uma fatia específica. O truque é pedir explicação ancorada em arquivo e linha, não explicação genérica — assim você consegue conferir.



```
onboarding

> explique o fluxo de login e cite
  os arquivos envolvidos

• Grep  "session" (14 arquivos)
• Read  src/auth/middleware.js
• Read  src/auth/tokens.js

O login passa por 3 camadas:
middleware → tokens → sessão.
```

Você ainda precisa abrir os arquivos citados e conferir. A diferença é que agora você sabe quais abrir.

CAPÍTULO

03

PONTA A PONTA

Funcionalidade completa, com testes

DESCREVA O COMPORTAMENTO, NÃO A IMPLEMENTAÇÃO

O erro mais comum é pedir código como se estivesse ditando: 'crie uma função que recebe X e retorna Y'. Isso desperdiça a parte mais útil da ferramenta, que é ela decidir a implementação a partir das convenções que já existem no seu projeto.

Descreva o comportamento esperado e os critérios de aceite. Deixe a estrutura por conta de quem já leu o repositório inteiro.

Um pedido bom descreve o resultado. Um pedido ruim descreve os passos.

E sempre peça os testes junto. Não por disciplina: porque um teste que passa é a única evidência de que a coisa funciona.

O CICLO QUE VALE A PENA

Na prática o fluxo vira um laço curto: você descreve, ela implementa e roda, você revisa o diff. Se o teste falha, ela mesma vê a saída e corrige antes de te devolver.

A terminal window with a dark blue background and light green text. The title bar at the top says 'feature' and has three colored window control buttons (red, yellow, green) on the left. The terminal content shows a sequence of actions: a prompt followed by a description of a feature, a list of steps with green bullet points, and the execution of 'npm test' which results in '1 failing'. This is followed by another list of steps, and a second execution of 'npm test' which results in '6 passed'.

```
feature

> usuário só vê os próprios pedidos.
  cubra com testes.

• Write tests/pedidos.test.js
• Edit  src/controllers/pedidos.js
• Bash  npm test
  1 failing
• Edit  src/controllers/pedidos.js
• Bash  npm test  6 passed
```

Repare no passo do meio: o teste falhou e foi corrigido sem você entrar no circuito. É aí que o tempo aparece.

CAPÍTULO

04

DEBUGGING

Da mensagem de erro à causa raiz

PARE DE PESQUISAR A MENSAGEM DE ERRO

Colar um stack trace no buscador te dá o caso genérico. O seu bug quase nunca é o caso genérico — ele é a interação entre a biblioteca e alguma decisão específica do seu projeto.

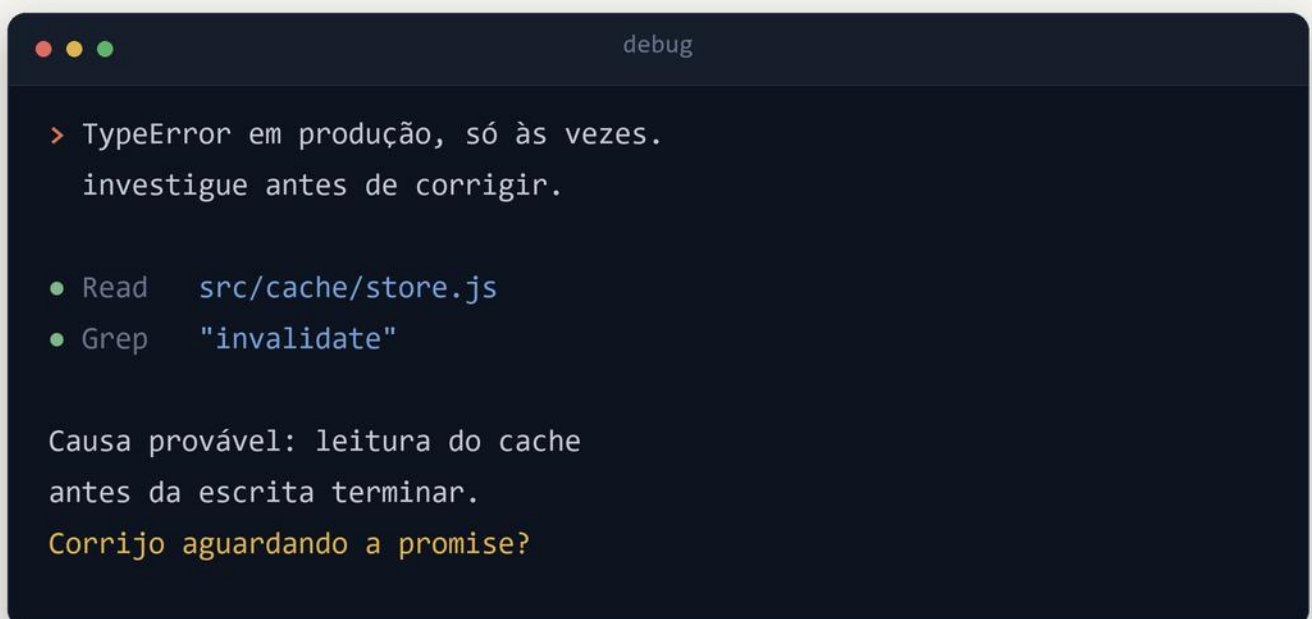
Uma ferramenta que lê o seu código investiga o seu caso. Ela segue a pilha até o arquivo real, olha o que está em volta e propõe uma hipótese verificável.

A pergunta certa não é 'o que significa esse erro'. É 'por que ele acontece aqui'.

Quando o erro não reproduz sempre, dê o contexto que só você tem: o que mudou, desde quando, em qual ambiente.

INVESTIGAR ANTES DE CORRIGIR

Vale um hábito: peça o diagnóstico antes da correção. Uma correção proposta sem explicação é uma aposta; com explicação, você consegue julgar se faz sentido.



```
debug  
  
> TypeError em produção, só às vezes.  
  investigue antes de corrigir.  
  
● Read   src/cache/store.js  
● Grep   "invalidate"  
  
Causa provável: leitura do cache  
antes da escrita terminar.  
Corrijo aguardando a promise?
```

Note o ponto de interrogação no fim. Ela para e pergunta — e é exatamente esse o comportamento que você quer.

CAPÍTULO

05

REFATORAÇÃO

Mudanças mecânicas, em escala

O TRABALHO CHATO É O MAIS FÁCIL DE DELEGAR

Renomear um conceito em 40 arquivos, migrar de uma biblioteca para outra, padronizar um jeito de tratar erro: tarefas que não são difíceis, são só longas. Ninguém aprende nada fazendo a trigésima substituição.

É o cenário ideal para delegar, com duas condições: que exista teste cobrindo o que está sendo mexido, e que você revise o diff em vez de confiar no relatório.

- Faça em lotes pequenos, não no repositório inteiro
- Commit separado da mudança de comportamento
- Se não há teste, escreva o teste primeiro

Refatoração sem teste não é refatoração. É reescrita com esperança.

A regra prática: se você não consegue revisar o diff, o lote está grande demais.

CAPÍTULO

06

MEMÓRIA DE PROJETO

Ensinando as regras do seu time

PARE DE REPETIR AS MESMAS INSTRUÇÕES

Se você corrige a mesma coisa toda sessão — 'use aspas simples', 'os testes rodam com pnpm', 'não mexe na pasta de migrations' — o problema não é a ferramenta. É que essa informação não está escrita em lugar nenhum.

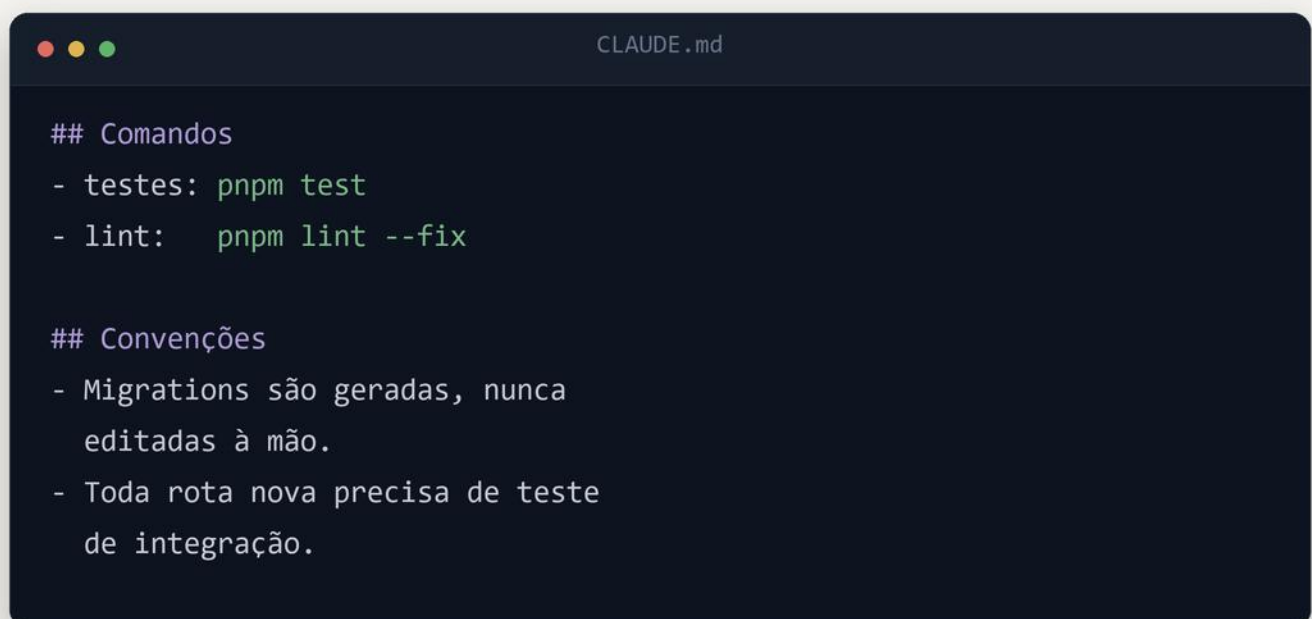
A solução é um arquivo de contexto no próprio repositório, lido automaticamente antes de qualquer ação. Funciona como o manual de onboarding que o time nunca escreveu, com uma vantagem: ele é versionado junto com o código, então nunca desatualiza em silêncio.

O que você explica duas vezes deveria estar escrito uma vez.

Comece pequeno. Três linhas úteis valem mais que duas páginas genéricas.

O QUE VALE A PENA REGISTRAR

Registre o que não é dedutível do código: comandos, convenções e proibições. Não registre o que a ferramenta descobre sozinha lendo os arquivos.



```
## Comandos
- testes: pnpm test
- lint: pnpm lint --fix

## Convenções
- Migrations são geradas, nunca
  editadas à mão.
- Toda rota nova precisa de teste
  de integração.
```

Esse arquivo tende a virar o documento mais honesto do projeto — porque é o único que alguém percebe quando está errado.

CAPÍTULO

07

ACELERADOR

Não substituto

O QUE CONTINUA SENDO TRABALHO SEU

Uma ferramenta que edita arquivos, roda comandos e cria commits tem poder de causar estrago. Isso não é motivo para não usar — é motivo para usar com as mesmas cautelas que você teria com qualquer pessoa nova no time.

- Revise o diff. Sempre. Não o resumo, o diff
- Cuidado redobrado com credenciais e CI/CD
- Decisão de arquitetura não se delega
- Se você não entende o código, ele não está pronto

O julgamento técnico continua inteiro do seu lado. O que muda é quanto do seu tempo sobra para exercê-lo, em vez de gastá-lo em tarefa mecânica.

Use IA como acelerador, nunca como dependência.

Quem sai na frente não é quem gera mais código. É quem consegue revisar bem o código que aparece.

FIM

Obrigado por ler até aqui

Este e-book nasceu de um desafio de projeto da DIO, na Formação ChatGPT for Devs, e é o segundo capítulo de uma dupla: o primeiro foi um artigo sobre o mesmo tema.

Todo o processo está aberto no repositório — os prompts usados em cada etapa, o script que gera este PDF e o artigo completo. Se você quiser produzir o seu, é só clonar e trocar o conteúdo.

github.com/filipemzanin

Se este material te ajudou, o melhor retorno possível é você publicar o seu.